EP 3 – Algoritmos em Grafos 2025.3 – UFABC

Prof. Aritanan Gruber

Entrega via Moodle até **08/12/2025 às 23h59**

Este Exercício Programa pode ser feito em C, C++ ou Python 3. Importante: seu arquivo fonte (em formato texto) deve-se chamar **cluster.[c|cpp|py]**, com a extensão apropriada à linguagem de programação escolhida; a não aderência a essa regra pode implicar na não correção do seu EP. Você será avaliado com relação à

(i) qualidade do seu código (inclua comentários essenciais explicando as partes principais e o porquê sua implementação resolve o problema; inclua um comentário com a análise de tempo da **sua** solução), e

(ii) a correção das respostas fornecidas após a execução (seu código será testado em instâncias que você não terá acesso; a pontuação não será binária, i.é, simplesmente baseada em correto / incorreto, mas levará em conta o quão longe uma resposta incorreta está da solução esperada); para que a correção funcione apropriadamente, você deve seguir minuciosamente os formatos de entrada e saída estipulados.

A descrição do problema a ser resolvido está na Seção 1, juntamente com os formatos de entrada (leitura da entrada padrão) e saída (escrita na saída padrão) a serem rigorosamente seguidos. O não cumprimento de tal contrato implicará na perda de pontos. Dicas de como resolver o problema são dadas na Seção 2.

1 Problema

Neste exercício programa, vamos focar em questões de viabilidade de um problema de escalonamento de tarefas em um cluster de máquinas idênticas que possuem preempção.

Especificamente, este cluster é composto por máquinas com o mesmo poder de computação operando em paralelo (o paralelismo é *grosso*, i.é, no nível de tarefas e não de instruções). Cada máquina do cluster pode processar uma única tarefa por vez, mas disponibiliza *preempção*: é possível interromper a execução de uma tarefa em uma máquina e retomá-la posteriormente, na mesma máquina ou em outra, do ponto em que a tarefa foi previamente interrompida (sem perdas). Vamos supor, a título de simplificação inicial, que cargas, pausas, mudanças de máquina e retomadas de tarefas — comummente referenciado por *troca de contexto* — pode ser feita instantaneamente (i.é, não há custos temporais envolvidos; modificações no modelo para acomodar custos em trocas de contexto são simples e não alteram sua estrutura).

Em cada instância do problema, são fornecidos um número $m > 0$ de máquinas do cluster ao que você terá acesso e um conjunto T com $n > 0$ tarefas, em que cada tarefa $t \in T$ possui:

- Um requisito de processamento p_t que denota o número de unidades de tempo necessárias para realizar tal tarefa;
- Um instante de liberação r_t, que representa a unidade de tempo a partir da qual a tarefa está disponível para processamento (ela poderia estar aguardando dados, por exemplo);
- E um valor $d_t \geq p_t + r_t$ que indica o primeiro instante, em unidades de tempo, em que a tarefa deve, impreterivelmente, ter sido completada. Isto é, a tarefa t deve ser realizada no intervalo $[r_t, d_t]$.

Cada tarefa possui uma natureza inerentemente sequencial e, dado o nível de paralelismo (grosso) do cluster, cada tarefa não pode ser processada em mais de uma máquina simultaneamente (não queremos esperdiçar trabalho computacional via duplicação ou algo pior).

Um escalonamento das n tarefas em m máquinas com preempção é **viável** se existe um *plano* de alocação das tarefas nas máquinas (ao longo do tempo) tal que todas as tarefas são executadas e completadas dentro de seus intervalos de tempo permitidos.

Você deve implementar um verificador de viabilidade para o problema acima. I.é, não estamos interessados, no momento, no plano de alocação / execução das tarefas nas máquinas, apenas na sua existência. Formatos de entrada e saída seguem.

Entrada

Seu escalonador deve estar preparado para trabalhar com diversas instâncias de entrada. Cada instância tem o formato que segue. Na primeira linha, são fornecidos os números de máquinas, $0 \leq m \leq 100$, e de tarefas, $0 \leq n \leq 1000$, respectivamente. Valores $m = 0$ e $n = 0$ indicam o término do processamento das instâncias (e não devem ser processados). Em caso contrário, nas próximas n linhas são fornecidos os valores $p_t \geq 0, r_t \geq 0$ e $d_t \geq 0$ (uma tripla por linha) para as tarefas $t \in T$. Os instantes r_t e d_t são inteiros, e p_t é racional fornecido em representação decimal. Todos os valores da entrada que constem em uma mesma linha são separados por um número qualquer de espaços em branco.

Saída

Para cada instância solucionada, você deverá imprimir um identificador `Instancia h`, em que `h` é um número inteiro, sequencial e crescente a partir de 1. Na linha seguinte deve ser impresso `Viavel` ou `Inviavel`, dependendo do escalonamento para a instância ser ou não viável, respectivamente.

Uma linha em branco deve separar a saída de cada instância.

Exemplo

Entrada

```
3 4
1.5  3 5
1.25 1 3
2.1  3 7
3.6  5 9
3 1
3 1 2
0 0
```

Saída

```
Instancia 1
Viavel

Instancia 2
Inviavel
```

2 Dicas e Info

Desta vez, temos um problema que pode ser resolvido via *st*-fluxos! E você deve solucioná-lo desta forma, implementando um algoritmo fortemente polinomial como Edmonds-Karp (ou outro assintoticamente melhor[1].

Cada máquina oferece $d_{\max} - r_{\min} + 1$ tempo para processamento, em que $r_{\min} := \min\{r_t : t \in T\}$ e $d_{\max} := \max\{d_t : t \in T\}$ (o '+1' advém dos tempos de liberação e *deadline* serem inteiros e do intervalo ser fechado). Logo, temos $\gamma = m*(d_{max} - r_{min} + 1)$ tempo total disponível. Dependendo da instância, subintervalos O_j de $F = [r_{\min}, d_{\max}]$ devem ficar ociosos: algumas tarefas devem terminar antes do início de um tal O_j e as demais ainda não podem começar. Refinando um pouco mais esta observação (as tarefas podem se sobrepor em segmentos de intervalos), chegamos à partição de $F \setminus \cup_j O_j$ em $I = \{I_1, I_2, \ldots, I_k\}$, em que $I_i = [b_i, e_i]$, $b_i < e_i < b_{i+1}$. Cada máquina tem $e_i - b_i + 1$ tempo de processamento em I_i, e b_i e e_i são *eventos*: releases ou deadlines de tarefas. Não é difícil perceber agora que o problema de viabilidade consiste em determinar se o tempo p_t de cada tarefa t pode ser distribuído em um subconjunto de intervalos em I que respeitem r_t e d_t. Em poucas palavras, o tempo é um fluxo, e a estrutura do digrafo impõe as restrições desejadas.

3 Integridade

O EP é individual e espera-se a aderência aos critérios de integridade acadêmica da UFABC. Se for detectado que seu código foi copiado de algum colega, livro, site na internet, escrito por alguma IA generativa, ou qualquer coisa que o valha (i.é, não escrito por você), você poderá ser chamado a dar explicações. Se ficar comprovada a ocorrência de uma trapassa, você receberá conceito **F** no curso (independente das demais avaliações, como combinado na primeira aula).

[1] Dinitz, com a modificação de Malhotra-Kumar-Maheshwari, costuma ser o mais rápido na prática, garante tempo assintótico $O(n^3)$ e requer somente um pouco mais de trabalho na implementação.